

Given a string consisting of only alphabets (either lowercase or uppercase). Print all the characters of string in such a way that for all lowercase character, print its uppercase character and for all uppercase character, print its lowercase character.

TestCase

Input

"Hello"

Output

"hELLO"

a b - - - z
↓ ↓ ↓
97 98 122

A B - - - Z
↓ ↓ ↓
65 66 90

```
→ int n = S.size();  
for (int i = 0; i < n; i++) {  
    if (s[i] >= 97) {  
        char c = s[i] - 32;  
        print(c);  
    } else {  
        char c = s[i] + 32;  
        print(c);  
    }  
}
```

⇒ How many substrings
in "abcd";

$$\Rightarrow \frac{n(n+1)}{2} = 10$$

- Check whether the given substring of string s is palindrome or not.

A palindrome is the sequence of characters that reads the same forward and backward. for example, "nayan", "madam", etc.

- TestCase

Input

```
s = "anamadamspe"
start = 3
end = 7
```

Output anamadamspe

```
true
```

bool isPalindrome (String s, int start, int end) {

while (start < end) {

if (s[start] != s[end])
return false;

else

start++; end--;

}

return true;

}

TC: $O(n)$

Given a string s , calculate the length of longest palindromic substring in s .

Test Case

Input

"anamadamm"

Output

5

Explanation

The substring "madam" of size 5 is the longest palindromic substring that can be formed from given string.

$\text{int max_len} = 1$

$\text{for}(\text{int } i = 0; i < n; i++)$ {

$\text{for}(\text{int } j = i; j < n; j++)$ {

$\text{if}(\text{isPalindrome}(s, i, j))$ {

$\text{int len} = j - i + 1;$

$\text{max_len} = \max(\text{max_len}, \text{len});$

 }

 }

$\text{return max_len};$

$T.C : O(n^3)$

0	1	2	3	4	5	6	7	8
m	a	d	a	m	a	n	a	n

→ Odd Length Palindrome.

0	1	2	3	4	5	6	7	8
m	a	d	a	m	a	n	a	n

max-len $\Rightarrow$ 1;

for (int i=0; i<n; i++) {

 l = r = i;

 while (l >= 0 && r < n) {

 if (s[l] != s[r])

 break;

 l--; r++;

 int len = r - l - 1;

 max-len = max(max-len, len);

}

⇒ Even length

0	1	2	3	4	5	6	7	8
m	a	d	a	m	a	n	a	n

max-len ⇒ 1;

for (int i = 0; i < n; i++) {

l = i, r = i + 1;

while (l ≥ 0 && r < n) {

if (s[l] != s[r])

break;

l--; r++;

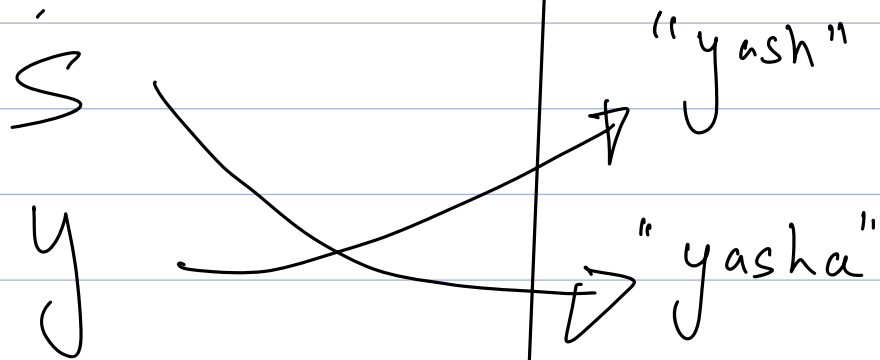
int len = r - l - 1;

max-len = max(max-len, len;

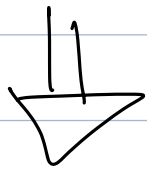
}

String $\textcircled{S} \Rightarrow$ "yash";

String y = "yash";



$S = S.append('a');$



$O(n)!$

String pool

String Builder.

hello:

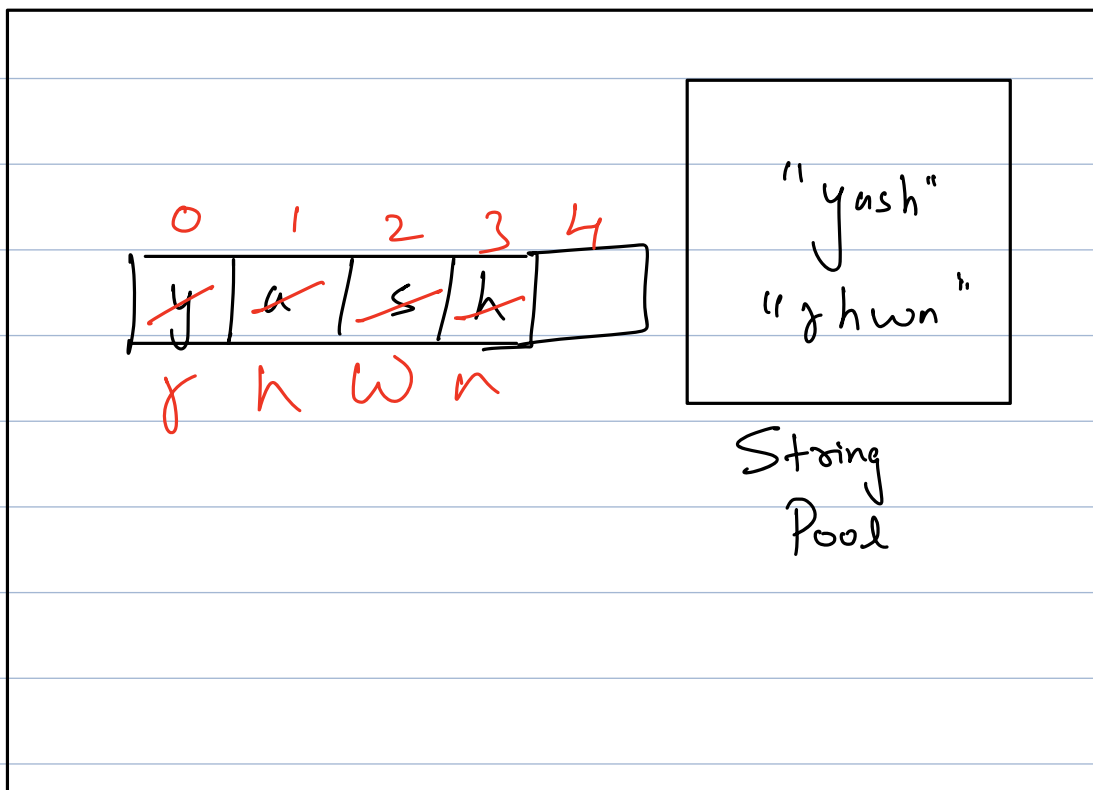
h | e | l | l | o | w

String Builder sb = new String Builder(s);

sb.append('w');

String ans = sb.toString();

return ans



SC: O(n)